Project Deep-Bluffing

Heads-Up Limit Hold'em Agent

A Monte Carlo Equity Estimation Approach

Nupur Sehgal

Roll Number: 250741

Abstract

This report describes the design and implementation of an autonomous agent for Heads-Up Limit Texas Hold'em (HULHE). The agent combines Monte Carlo simulation for hand-equity estimation with a rule-based decision layer that incorporates pot-odds reasoning, hand-strength tiering, and a bounded bluffing frequency. We describe the game engine architecture, justify the state representation and decision heuristics mathematically, and report empirical validation results, including a hand-evaluator optimization that yields a 13.9x speedup verified against 70,000+ randomly generated hands with zero discrepancies.

Contents

1 Introduction

Heads-up poker is a game of imperfect information: unlike the perfect-information setting of earlier control tasks, a player must act without observing the opponent's private cards. Our objective is to build an agent that estimates its own hand strength under this uncertainty and converts that estimate into a betting decision consistent with the fixed-limit betting structure specified for this tournament.

Our overall approach has two stages:

1. **Equity Estimation.** Given the current hole cards and community cards, estimate the probability of winning at showdown against a single random opponent hand, using Monte Carlo simulation.
2. **Decision Making.** Convert that equity estimate, together with pot odds and the current betting street, into a discrete FOLD / CALL / RAISE decision using a rule-based policy.

This design was chosen over alternatives such as Counterfactual Regret Minimization (CFR) or a Deep Q-Network (DQN) for reasons discussed in Section 7.

2 System Architecture

The codebase is organized into the following modules:

- `constants.py` — shared constants (ranks, hand categories, blind/bet sizes, betting cap).
- `card.py` — an immutable `Card` object with rank/suit validation and comparison dunder methods.
- `deck.py` — a 52-card deck with shuffling and dealing primitives.
- `player.py` — per-player state: stack, hole cards, current bet, fold status, and a reference to the decision-making bot instance.
- `hand_evaluator.py` — classifies any 5-card hand into one of the 10 standard categories and finds the best 5-card hand out of 7 cards (Section 4).
- `game_engine.py` — orchestrates a full isolated hand: blind posting, dealing, alternating betting rounds, showdown, and pot distribution (Section 3).
- `monte_carlo.py` — equity estimation via repeated random simulation (Section 5).
- `poker_bot.py` — the rule-based decision policy (Section 6).

- `agent.py` — the `BasePokerBot`/`CustomPokerBot` entry point required by the tournament arena, gluing the above together.

3 Game Engine Design

3.1 State Representation

Each hand is treated as an isolated episode: both players' stacks are reset to 100 chips at the start of every hand, consistent with the specification. The engine tracks, per hand: the community cards revealed so far, the pot size, the current street, the number of bets placed on the current street (for cap enforcement), and which player was the last aggressor (for betting-round termination).

3.2 Turn Order

Heads-up poker has an asymmetric turn order that flips after the first betting round:

- **Pre-flop:** the Small Blind (dealer) acts first.
- **Post-flop (Flop/Turn/River):** the Big Blind acts first.

This is implemented directly in `game_engine.py` by branching on the current street when selecting the first actor for a betting round.

3.3 Betting Round Termination

A betting round is closed under one of two conditions:

- **No raise has occurred this street:** action closes once both players have acted once (i.e., control returns to whoever acted first).
- **A raise has occurred:** action closes once control returns to the player who made the last raise (i.e., their raise has been matched or folded to).

This is implemented with an explicit `last_aggressor` pointer that is set on every `RAISE` action and checked at the end of every turn.

3.4 Fixed-Limit Bet Sizing and the Betting Cap

The raise increment is determined by the current street:

$$\text{raise\_increment}(\text{street}) = \begin{cases} 2 & \text{street} \in \{\text{Pre-Flop}, \text{Flop}\} \\ 4 & \text{street} \in \{\text{Turn}, \text{River}\} \end{cases}$$

The cap rule limits each street to at most 4 total bets (1 initial bet + 3 raises). We enforce this with a per-street counter, seeded to 1 pre-flop (since the Big Blind's forced post already constitutes the street's first "bet") and 0 on every subsequent street (since the first aggressive action on Flop/Turn/River is itself the street's first bet). A `RAISE` action is only offered in `legal_actions` while this counter is below the cap of 4.

3.5 Split Pots

When both hands are of identical rank and tiebreaker value at showdown, the pot is split evenly. Since blinds sum to an odd number of chips (1 + 2 = 3), pots are frequently odd-valued; the single remainder chip after integer division is awarded to the non-dealer player for that hand, so that this minor bias rotates with the button across a match rather than being fixed to one player.

3.6 Robustness to Short Stacks

Rather than raising an exception when a requested bet exceeds a player's remaining stack, all chip commitments are capped at the player's current stack. This prevents a legitimate (if rare) short-stack situation from crashing the engine mid-tournament, which is explicitly penalized under the grading rubric's stability criterion.

4 Hand Evaluation

4.1 Five-Card Classification

Given five cards, we compute:

- rank-count groups via `collections.Counter`, sorted by (count, rank) descending, which handles quads/full-house/trips/two-pair/ pair classification and their kicker ordering uniformly;

- a flush check (all five cards share a suit);
- a straight check over the five distinct rank values, including the special-cased "wheel" straight (A-2-3-4-5), whose high card is conventionally 5 rather than 14 despite the Ace's numeric value of 14.

These are combined into one of the 10 standard categories, each represented as a `HandScore` object holding an integer category and a tiebreaker tuple. Two hands are compared lexicographically on (category, tiebreakers), which is correct because category strictly dominates tiebreaker value by the rules of poker (e.g., any flush beats any straight, regardless of the specific cards involved).

4.2 Seven-Card Evaluation: An Optimization

The Texas Hold'em showdown requires finding the best 5-card hand out of the 7 available cards (2 hole + 5 community). A direct brute-force approach evaluates all $\binom{7}{5} = 21$ five-card subsets and keeps the maximum. This is correct, but wasteful: many of the 21 subsets share overlapping information (e.g., the same flush suit, the same rank counts), which the brute-force approach recomputes from scratch 21 times.

We instead evaluate the 7 cards directly in a constant number of passes:

1. Count ranks across all 7 cards once.
2. Identify a flush suit, if any (at most one suit can appear ≥ 5 times among 7 cards, since $5 + 5 > 7$).
3. If a flush suit exists, search for a straight *within* that suit's ranks (straight flush / royal flush).
4. Otherwise, search for a straight across all 7 cards' distinct ranks.
5. Classify using the rank-count groups, handling two edge cases that do not arise in the 5-card case but can arise in 7 cards: two distinct three-of-a-kinds (resolved as a full house using the higher trip as the triple and the lower trip's rank as the pair), and three or more pairs (resolved by taking the two highest-ranked pairs).

This produces identical `HandScore` results to the brute-force approach, but with substantially less repeated work.

4.3 Empirical Validation

Because a hand-evaluator rewrite is exactly the kind of change that can appear correct while being subtly wrong on rare hand types, we validated the optimized evaluator against

the original brute-force implementation using two complementary methods:

- **Randomized testing:** 70,000 randomly dealt 7-card hands were evaluated by both implementations; category and tiebreaker tuples were required to match exactly. **Result: 0 mismatches.**
- **Targeted edge-case testing:** hands specifically constructed to exercise the two edge cases described above (two three-of-a-kinds; three or more pairs), plus a wheel straight flush, a full 7-card flush, and four-of-a-kind kicker selection among three remaining cards. **All passed.**

4.4 Performance Impact

Implementation	Time / hand evaluation	Relative speed
Brute force (21 combinations)	116.7 µs	1.0x
Direct evaluation	8.4 µs	13.9x

Table 1: Per-call timing, averaged over 2000 randomly dealt 7-card hands.

Since Monte Carlo simulation (Section 5) calls the seven-card evaluator twice per simulated outcome (once for the agent's hand, once for the simulated opponent's hand), this optimization compounds: the full decision pipeline's measured cost per hand dropped from approximately 302 ms to 39.7 ms (a 7.6x end-to-end speedup), reducing the projected wall-clock time for a 10,000-hand match from roughly 50 minutes to under 7 minutes.

5 Monte Carlo Equity Estimation

5.1 Method

Given hole cards H and the current community cards C (with $|C| \in \{0, 3, 4, 5\}$), we estimate our equity by repeated simulation:

1. Remove $H \cup C$ from a fresh 52-card deck, leaving the pool of unseen cards.
2. For each of N trials:
   (a) Randomly deal 2 cards to a hypothetical opponent from the unseen pool.
   (b) Randomly complete the community board to 5 cards using the remaining unseen pool (if not already complete).

(c) Evaluate both 7-card hands and record a win, tie, or loss.

3. Aggregate: $\hat{p}_{\text{win}}, \hat{p}_{\text{tie}}, \hat{p}_{\text{loss}}$ as the respective sample frequencies over the N trials.

5.2 Equity Formula

A tie returns half the pot rather than none of it, so in expectation a tie is worth exactly half of a win. We therefore define equity as

$$\text{equity} = \hat{p}_{\text{win}} + \tfrac{1}{2}\,\hat{p}_{\text{tie}}$$

which is the quantity consumed by the decision policy (Section 6), rather than $\hat{p}_{\text{win}}$ alone.

5.3 Choice of Sample Size

Monte Carlo estimation error scales as $O(1/\sqrt{N})$. Rather than default to a large, uniform simulation count on every decision, we selected the per-street simulation count empirically by comparing lower-N equity estimates against a high-precision ($N = 5000$) reference on a representative flop scenario:

N (simulations)	Avg. absolute equity error	Time per estimate
50	0.070	12.4 ms
100	0.043	24.5 ms
150	0.038	34.5 ms
200	0.041	45.1 ms

Table 2: Equity estimation error vs. simulation count, averaged over 20 trials at each N, against a 5000-simulation reference.

Because the decision policy only needs to place the equity estimate into the correct hand-strength tier (Section 6), and adjacent tiers are separated by a 0.20 gap, an average absolute error of 0.04–0.07 is rarely sufficient to cause a tier misclassification. We therefore use substantially fewer simulations than a naive high-precision choice would suggest:

$$N(\text{street}) = \begin{cases} 100 & \text{Pre-Flop} \\ 150 & \text{Flop, Turn} \\ 200 & \text{River} \end{cases}$$

This is a deliberate precision/speed trade-off, justified by the coarseness of the downstream decision rule rather than an accuracy requirement in its own right.

6 Decision Policy

6.1 Hand-Strength Tiering

The equity estimate is mapped to one of five discrete tiers via fixed thresholds:

Tier	Equity range
VERY_WEAK	$[0, 0.25)$
WEAK	$[0.25, 0.45)$
MEDIUM	$[0.45, 0.65)$
STRONG	$[0.65, 0.85)$
MONSTER	$[0.85, 1.0]$

6.2 Pot Odds

The classical profitable-call condition from the assignment's EV formula is

$$EV_{\text{call}} = [p_{\text{win}} \times \text{pot}] - [(1 - p_{\text{win}}) \times \text{amount\_to\_call}] \geq 0$$

which rearranges to the equivalent and more convenient pot-odds condition:

$$p_{\text{win}} \geq \frac{\text{amount\_to\_call}}{\text{pot} + \text{amount\_to\_call}} = \text{pot\_odds}$$

MEDIUM and WEAK hands continue only when this condition holds; STRONG and MONSTER hands continue (and raise) unconditionally, since their equity is high enough that the pot-odds condition is virtually always satisfied in practice; VERY_WEAK hands never pay to continue.

6.3 Bluffing

A purely equity-driven policy is exploitable: an opponent who observes that every raise corresponds to a strong hand can profitably fold to every raise and call every check. To avoid this, the policy raises with a small probability (5%) even on VERY_WEAK or WEAK hands, restricted to the Pre-Flop and Flop streets (i.e., while there are still cards to come and the bluff has room to represent a hand that improves). This was empirically verified to fire at the intended $\approx 5\%$ rate on both eligible streets (112/2000 and 108/2000 trials respectively in isolated testing).

6.4 Decision Table

Tier	Action
MONSTER	Raise if legal, else call.
STRONG	Raise if legal, else call.
MEDIUM	Call/raise if $p_{\text{win}} \geq$ pot_odds, else fold.
WEAK	Check for free; call if pot-odds justify it; else fold.
VERY_WEAK	Check for free; else fold.

A final safety fallback guarantees that a legal action is always returned even in unanticipated states, defaulting to CALL where legal and FOLD otherwise.

7 Alternative Approaches Considered

- **Counterfactual Regret Minimization (CFR).** CFR converges to a Nash equilibrium strategy and would reason about opponent ranges directly rather than assuming a uniformly random opponent hand. We did not pursue it primarily due to the implementation complexity of information-set abstraction within the assignment's time budget, and because our Monte Carlo approach is easier to verify empirically (Section 4), which we prioritized given the correctness-focused grading criteria.
- **Deep Q-Network (DQN).** A DQN could in principle learn a policy directly from self-play, but requires careful reward shaping, a hidden-state-aware architecture (since raw hole cards and hidden opponent information make this a POMDP rather than a standard MDP), and substantial training infrastructure. Given the

fixed-limit, heads-up structure of this tournament, we judged the added complexity unlikely to outperform a well-tuned equity-based heuristic within the available development time.

- **Monte Carlo Tree Search (MCTS).** MCTS over the betting tree was considered, but the fixed-limit structure already bounds the action space tightly (at most 3 discrete actions per decision, capped raises), which reduces MCTS's main advantage (handling large branching factors) relative to its implementation cost here.

Our approach is best understood as a fast, verifiable approximation to the EV formula given in the assignment: rather than solving for an equilibrium strategy, we estimate the numerator terms of the EV formula directly via simulation and act greedily (with a bounded bluff perturbation) on the result.

8 Known Limitations

- **Uniform opponent modeling.** The Monte Carlo simulation assumes the opponent's hole cards are uniformly random from the unseen deck. In reality, an opponent's prior actions (e.g., multiple raises) should narrow the range of hands they are likely to hold; our current implementation does not condition on betting history when simulating the opponent's hand.
- **Static thresholds.** The hand-strength tier boundaries and the 5% bluff frequency are fixed constants rather than learned or adapted to a specific opponent's tendencies over the course of a match.
- **No explicit bet-sizing strategy.** Bet sizes are fully determined by the fixed-limit structure enforced by the arena, so this limitation is inherent to the game format rather than a gap in our agent, but is noted for completeness.

9 Empirical Validation Summary

The full pipeline (engine, evaluator, Monte Carlo estimation, decision policy, and the `agent.py` entry point) was validated via:

- 55 unit-level checks covering card comparison, deck integrity, all 10 hand categories (including the wheel straight and wheel straight flush edge cases), split-pot tie detection, player betting mechanics, and engine-level scenarios (check-check to showdown, immediate fold, a full raise war to the betting cap, dealer alternation).

- 500 hands played with uniformly random legal actions, confirming no crashes and exact chip conservation ($\text{stack}_1 + \text{stack}_2 = 200$) on every hand.
- 300 hands played end-to-end using the actual `agent.CustomPokerBot` class exactly as the tournament arena will instantiate and call it, confirming no crashes, exact chip conservation, and a per-hand runtime consistent with completing a 10,000-hand match well within a reasonable time budget.
- 70,000+ randomly generated hands plus targeted edge cases confirming the optimized hand evaluator (Section 4) is exactly equivalent to the original brute-force implementation.

10 Conclusion

We implemented a complete Heads-Up Limit Hold'em game engine from scratch, enforcing the specified turn order, fixed-limit bet sizing, and 4-bet cap, and built an autonomous agent that estimates hand equity via Monte Carlo simulation and converts that estimate into betting decisions using pot-odds reasoning, hand-strength tiering, and a bounded bluffing frequency. We additionally identified and resolved a significant performance bottleneck in hand evaluation, yielding a verified 13.9x per-evaluation speedup with zero observed correctness regressions across extensive randomized and targeted testing. The resulting agent completes a 10,000-hand match in well under ten minutes, comfortably within the constraints of a round-robin tournament.